Performance Evaluation of TinyLlama: Effects of Temperature and Concurrency

1 Introduction

In this report I evaluate the performance of a locally deployed tiny language model when varying the temperature and the number of users concurrently using the LLM system. The main focus is the effect of temperature on response behavior, especially response time and generated output length. User concurrency is included as a stress dimension to distinguish temperature-related effects from queueing effects. Experiments were performed on a BEAGLE-AI64 NPU accelerated board using Ollama and the `Tinyllama` model. The results show that higher temperatures mainly increase the number of generated tokens, while the generation time per token remains almost constant. Concurrency has a stronger effect on latency, since additional users mostly create queueing rather than proportional throughput gains.

2 Performance Model

The benchmark is modelled as a closed workload. A fixed number of users, named N, repeatedly sends prompts to the local LLM server. In each round, all users submit one request. Once the responses for the round are completed, the next prompt is submitted. The think time is therefore set to $Z = 0$. This is a closed workload because the number of active users is fixed and controlled by the experiment. The benchmark does not impose a predefined external arrival rate; instead, the request rate emerges from the interaction between the fixed user population and the system response time. The arrivals are bursty, since all users submit their prompt at the beginning of each round, but the workload remains closed because the population of users is constant throughout each experiment.

For a closed system, the relationship between the number of users N, the throughput X, the response time R, and the think time Z is:

$$N = X(R + Z). \tag{1}$$

Since the experiments use zero think time, this becomes:

$$R = \frac{N}{X}. \tag{2}$$

This relation is useful for interpreting the concurrency results. If the number of users increases but throughput does not increase proportionally, the response time must increase. In queueing terms, this indicates that additional requests are waiting for a limited service resource.

In our setup, Ollama was used with its default request-level parallelism configuration. Therefore, for a given loaded model, the server effectively processed one generation request at a time, while simultaneous requests waited in a queue. From the point of view of request execution, the system can therefore be approximated as a single-server queue. This assumption is important for interpreting the results: increasing the number of users *should* mainly increase queueing delay rather than providing proportional throughput improvement.

3 Metrics

Table 1 summarizes the metrics used in the evaluation. With these metrics we try to isolate the different phenomena causing performance variation. Changing the temperature may change the number of tokens generated for each answer, impacting the response time and thus the throughput. At the same time, if response time increases without an increase in output length, then the cause may be queueing, decoding inefficiency, or other system-level overhead.

Metric	Meaning
Response time	End-to-end user-perceived latency.
Output tokens	Number of tokens generated by the model.
Time per token	Generation duration divided by output tokens.
Response throughput	Completed responses per second.
Token throughput	Generated tokens per second.

Table 1. Different metrics we used to measure performance.

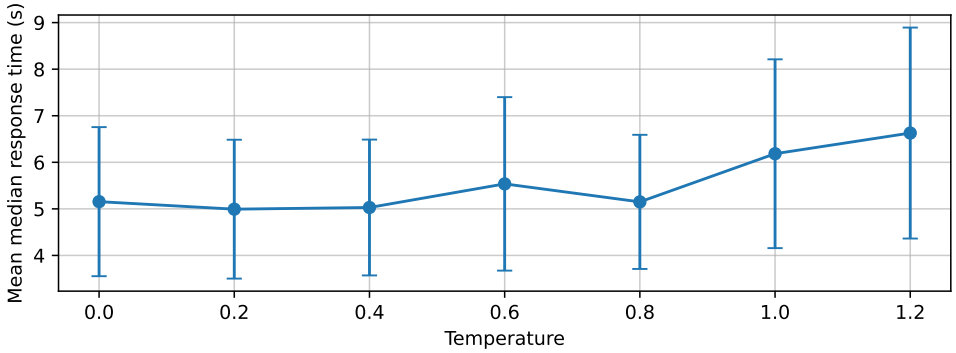

Fig. 1. Mean median response time vs temperature, averaged across user counts. Error bars show variance.

4 Experimental Setup

The experiments were done on a BEAGLE-AI64 board that was running Ubuntu 22.04. We used Ollama as the inference engine and it was serving the `TinyLlama` model. The prompts that were used came from the Dolly 15k instruction dataset. [0.0, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2.]

We changed the number of users at the time from 1 to 6. For each setup every user sent 60 requests. So a setup with a number of users generated a certain number of responses, which was 60 times the number of users. To make sure the prompts were not affecting the results all users asked the prompts in the same order. This way we knew that any differences between the setups were not because of the prompts being lengths or having different levels of difficulty. The TinyLlama model was set to generate no more than 250 tokens as the output. This was done to prevent some responses from causing problems with the experiment. (I did so because I found out that some questions could generate responses that were way too long, more than 10 thousand tokens. This would slow down the experiments. Even stop them from working).

5 Results

5.1 Temperature-Centered Analysis

Figure 1 shows the mean median response time as a function of temperature, averaged across user counts. The error bars represent the standard deviation across user counts. Response time remains relatively stable up to approximately $\theta = 0.8$. At higher temperatures, especially $\theta = 1.0$ and $\theta = 1.2$, response time increases. This suggests that high temperatures create additional work for the system, but the effect becomes visible mainly at the upper end of the tested range.

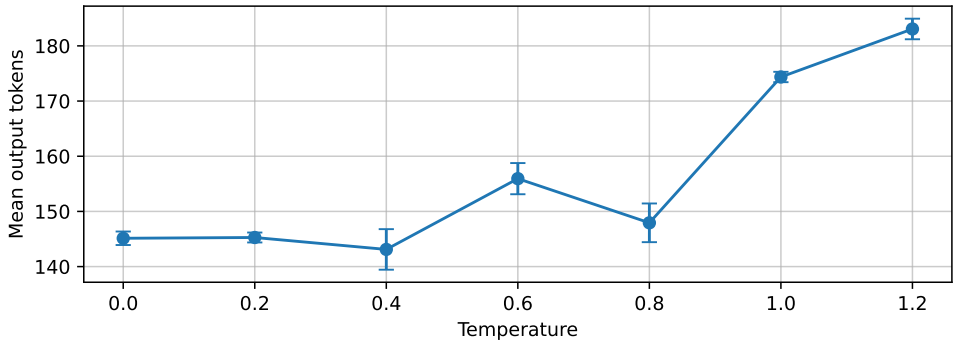

Fig. 2. Mean output tokens versus temperature, averaged across user counts. Error bars show standard deviation across user counts.

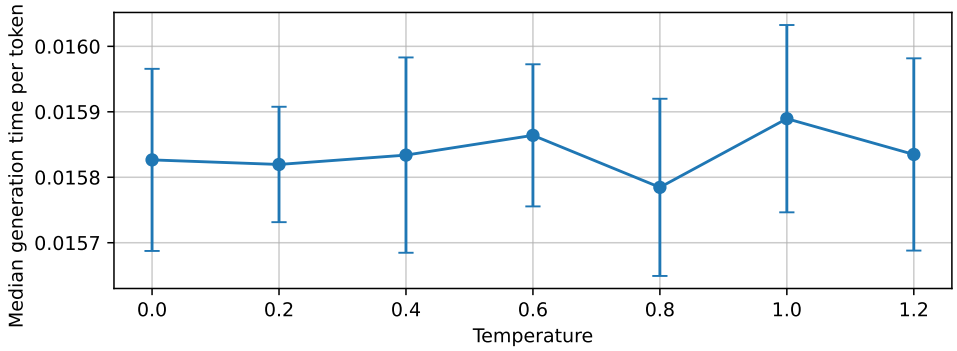

Fig. 3. Median generation time per token versus temperature, averaged across user counts.

Figure 2 shows the mean number of generated output tokens versus temperature. The token count increases more clearly than the response time. Low and medium temperatures produce similar output lengths, while temperatures 1.0 and 1.2 produce noticeably longer answers. This indicates that temperature mainly affects the amount of generated text.

Figure 3 shows the generation time per token. The generation time per token is almost constant across temperatures. Therefore, changing temperature does not significantly change the cost of generating each token. The increase in total response time at high temperature is mainly explained by longer responses.

5.2 User-Centered Analysis

Concurrency is another elemnt affecting how busy the servers are and how long people have to wait for a response. Figure 4 shows what happens to the response time when you have more users and we looked at this across different temperatures. The time it takes to get a response gets relevantly longer when you have users. This is what you would expect to happen when the servers are busy and cannot handle all the requests from users at thetime. In our setup Ollama can only handle one request from users at a time. So when multiple users send in requests, at the time Ollama works on one request and the others have to wait. As the number of users increases the wait time gets longer. That is why it takes longer to get a response. Concurrency and the number of users are really important here. Concurrency is what is causing the response time to increase because the number of users is making the queueing delay become larger.

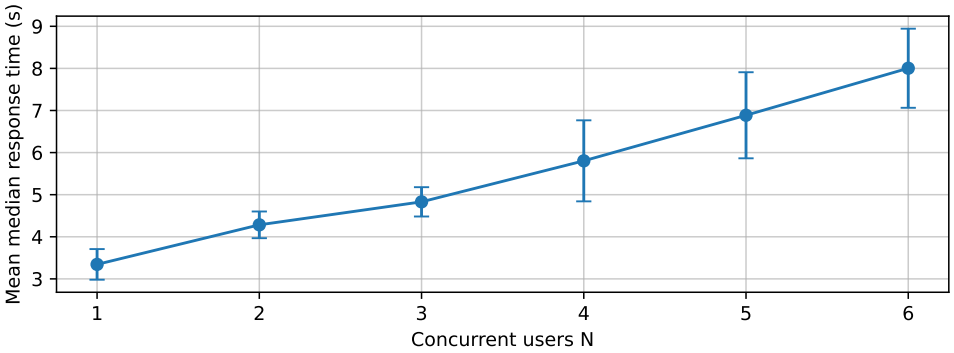

Fig. 4. Mean median response time versus number of users, averaged across temperatures.

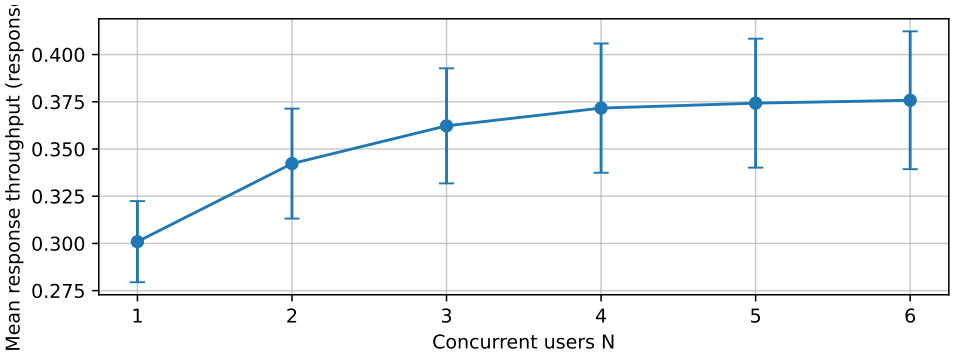

Fig. 5. Mean response throughput versus number of users, averaged across temperatures.

Figure 5 shows how response throughput changes depending on the number of active users. Interestingly, throughput also increases slightly as the number of users grows. This does not contradict the fact that Ollama processes one request at a time. We did not study the reason behind this phenomena, but a possibility is that with a single user, the server may experience small idle intervals between consecutive requests due to client-side scheduling. With multiple users, several requests are already waiting in the queue, so the server can start the next request immediately after completing the previous one. In other words, increasing the number of users may improve server utilization. However, the throughput gain is modest compared with the increase in response time. This indicates that the system is approaching the saturation throughput of the bottleneck resource. Once the server is almost continuously busy, adding more users cannot produce proportional throughput improvement; it mainly increases waiting time.

Token throughput behaves similarly to response throughput. It increases slightly with more users because the server is kept busy for a larger fraction of the experiment. Nevertheless, the improvement is much smaller than the increase in response time. Therefore, adding users increases utilization up to the bottleneck limit, but after that point it mostly increases queueing delay rather than useful completed work.

5.3 Interaction Between Temperature and Concurrency

The aggregated plots are useful, but they hide interactions between temperature and concurrency. Figure 7 shows median response time versus temperature with one curve per user count. The curves are mainly separated by the

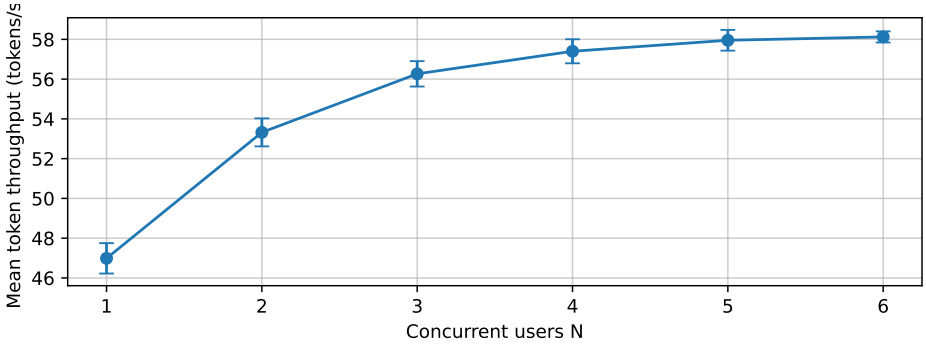

Fig. 6. Mean token throughput versus number of users, averaged across temperatures.

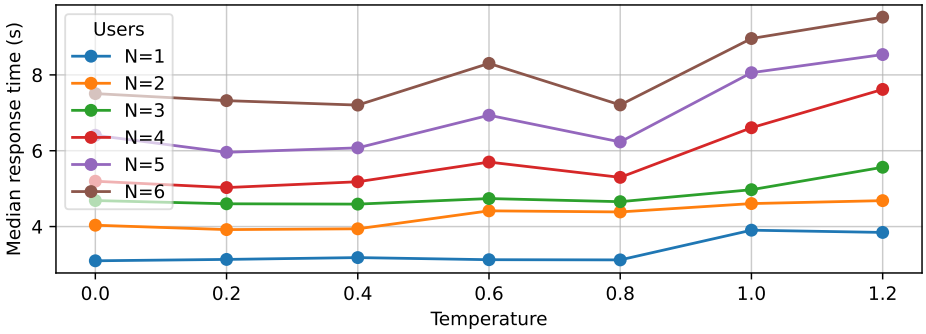

Fig. 7. Median response time versus temperature for each number of users.

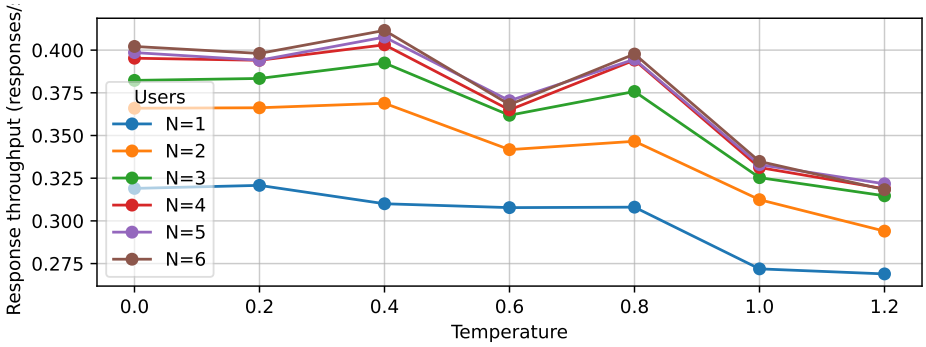

Fig. 8. Response throughput versus temperature for each number of users.

number of users, meaning that concurrency is the dominant factor for response time. However, high temperatures increase latency more clearly when there are more users. Longer responses occupy the server for longer, which increases the queueing delay of other users.

Figure 8 shows response throughput versus temperature for each user count. Throughput tends to decrease slightly at higher temperatures, especially where output length increases. This is expected: if each response contains more tokens, each request requires more service time, and fewer responses can be completed per second.

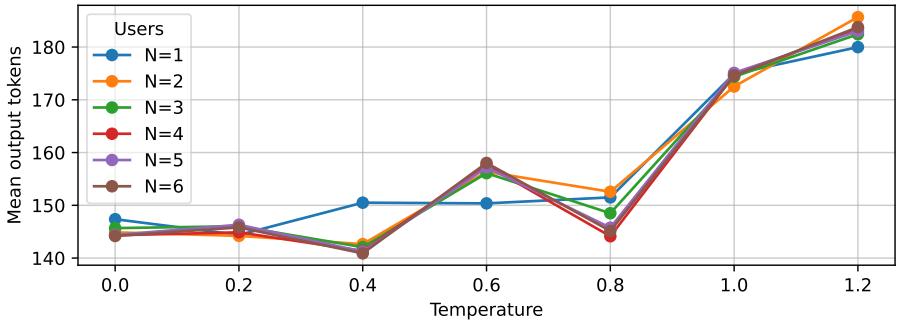

Fig. 9. Mean output tokens versus temperature for each number of users.

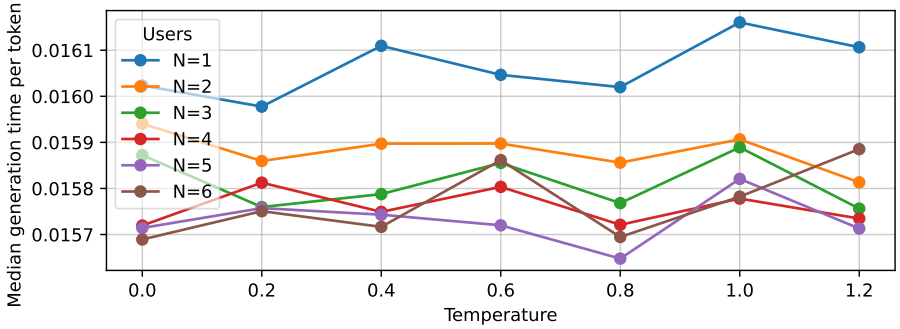

Fig. 10. Median generation time per token versus temperature for each number of users.

The curves are close to each other, Figure 9 shows output tokens versus temperature for each user count. showing that concurrency does not strongly affect output length. Output length is controlled mainly by temperature.

Figure 10 shows generation time per token. This figure confirms that per-token generation speed is stable. The main latency increase at high temperature is explained by longer outputs rather than slower token generation.

6 Conclusion

The experiments show that sampling temperature mainly affects the number of tokens generated by the model. Low and medium temperatures produce similar response times and output lengths, while high temperatures generate longer answers and modestly increase latency. The generation time per token remains nearly constant, so the model is not slower per token at higher temperature. Concurrency has a stronger effect on response time than temperature. As the number of users increases, response time increases significantly, while throughput improves only slightly. This indicates that the system is bottlenecked by request-level serving: additional users mostly create queueing rather than proportional throughput gains. Overall, temperature tuning mostly affects output length and response style, while serving scalability depends more on request parallelism or batching. For embedded systems such as Jetson boards, improving concurrency support is therefore more important than changing temperature if the objective is to reduce latency under multiple users.